GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 21 OF 42 | CORE 90 CHAPTER TOPIC

Core 90 - Streaming, scale and concurrency

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Streaming, scale and concurrency | 1 questions | tap a title to jump to it

M 1. Find top/bottom-k words in list or stream PracHub 2026-01-11

STUDY PATH: question 89 of 167 (Core 90)

Priority score 2.3 | Streaming, scale and concurrency | asked 282 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 1 of 1

Find top/bottom-k words in list or stream

POSTED 2026-01-11 | 282 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 117/228

ABOUT THIS QUESTION

Topic: Core 90 - Streaming, scale and concurrency

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words

LeetCode match: LC 692 Top K Frequent Words (+ stream variant)

Posted on PracHub: 2026-01-11 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/find-top-bottom-k-words-in-list-or-stream

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.
Constraints
0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated

QUESTION

EXAMPLES

```
Input: (['i', 'love', 'leetcode', 'i', 'love',
    'coding'], 2)
Expected Output: [['i', 'love'], ['coding',
    'leetcode']]
Explanation: `i` and `love` both appear twice, so
    lexicographical order gives `['i', 'love']`.
    `coding` and `leetcode` both appear once, so
    lexicographical order gives `['coding',
    'leetcode']`.
Input: (['apple', 'banana', 'apple', 'cherry',
    'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'],
    ['cherry', 'date', 'apple']]
```

Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.

Hints
Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).
Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.
Constraints
1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested
Examples
Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]
Expected Output: [['apple', 'banana'], ['banana', 'apple']]
Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.

Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]
Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]
Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.
Hints
Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

APPROACH

List: counts + heaps (min-heap size k for top, max-heap size k for bottom). Stream: keep counts in a map and maintain heaps with lazy updates, or approximate with Space-Saving / Count-Min when memory is bounded.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how ties break, whether top and bottom are over distinct words, what topK(k) returns for k larger than the number of words, and whether memory is bounded for the stream.

BRUTE FORCE TO SAY OUT LOUD

Count words, sort all distinct words by (count, word), take the first and last K. O(d log d).

OPTIMAL IDEA

List version: Counter, then heapq.nlargest and nsmallest with the key (count, word) in the right directions, O(n + d log K). Stream version: ingest increments a dict in O(1); topK and bottomK compute on demand with a heap in O(d log K). If queries are frequent, keep words grouped by count in buckets (a linked list of count buckets like an LFU cache) so topK walks from the highest bucket in O(K). If memory is bounded, the Space-Saving or Count-Min sketch approximates the top K; bottom-K cannot be approximated well, so say so.

DRY-RUN THESE CASES

Fewer words than K, all counts equal (ties decide everything), one word repeated, an empty stream.

LIKELY FOLLOW-UPS

Windowed counts (expire old words), or distributed counting with a merge step.

PITFALLS

Mixing up the direction for the bottom-K heap. Missing a deterministic tie-break.

HOW TO THINK ABOUT IT

Count first; then you only need the K best and K worst, which a heap does without sorting everything. For a stream, ingesting is just a counter increment, and the answer is computed from the counts on demand. If memory is bounded, only heavy hitters (top-K) can be approximated; the rarest words cannot.

Basis: LeetCode 692 (Top K Frequent Words) and its bottom-K counterpart, plus a streaming interface. Ties break alphabetically.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words, k):
    counts = Counter(words)
    def rank(w):
        return (-counts[w], w)
    ranked = sorted(counts, key=rank)
    most = ranked[:k]
    least = ranked[::-1][:k]
    return most, least
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct words, O(d log d). Two heaps of size K are O(d log k); a stream only needs the counts, and a Space-Saving sketch bounds memory for the top K.

OPTIMAL SOLUTION · Counter plus two heaps of size K

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream is too big to count every distinct word.

Use the Space-Saving (or Misra-Gries) summary with m counters: a word that appears more than n / m times is guaranteed to be tracked, and its stored count is an upper bound. This works for the top K only; the least frequent words cannot be found this way.

FOLLOW-UP 2 topK must be O(k), not O(d log k), because it is called all the time.

Keep words in buckets by count (a linked list of count buckets, like the LFU cache structure); the most frequent words are in the last bucket. An increment moves a word one bucket up in O(1).

FOLLOW-UP 3 Case and punctuation.

Normalise before counting (lowercase, strip punctuation) and say so; the counting and ranking code is unchanged.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find top/bottom-k words in list or stream
#
# You are given words (strings) either as a finite list or
# as an unbounded stream. 1) **List version**: Given an
```

```
# array words and an integer k, return: - the k most
# frequent distinct words, and - the k least frequent
# distinct words. Specify how you break ties (e.g.,
# lexicographically, or any order). 2) **Stream version
# (online)**: Design a data structure that supports: -
# ingest(word): process the next word in the stream -
# topK(k): return the k most frequent distinct words seen
# so far - bottomK(k): return the k least frequent
# distinct words seen so far Discuss time/space complexity
# and how you would implement these operations
# efficiently. 3) **Variant (discuss/design)**: Extend the
# stream design to support returning the **first k**
# and/or **last k** words that are currently **non-
# repeating** (i.e., words with frequency exactly 1)
# according to stream arrival order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream is too big to count every distinct word.
# 2. topK must be O(k), not O(d log k), because it is
# called all the time.
# 3. Case and punctuation.
#
# INTERVIEW NOTES
# Count first; then you only need the K best and K worst,
# which a heap does without sorting everything. For a
# stream, ingesting is just a counter increment, and the
# answer is computed from the counts on demand. If memory
# is bounded, only heavy hitters (top-K) can be
# approximated; the rarest words cannot.
#
# BRUTE FORCE (say it first): Sort all distinct words by
# (count, word)
# Count, sort the distinct words by count and then
# alphabetically, and take the first or last K. Correct
# and the oracle. Time O(d log d). Space O(d).


import heapq
from collections import Counter, defaultdict
```

```
# BRUTE FORCE (say it first): Sort all distinct words by
# (count, word) | time O(d log d)
def top_bottom_sort(words, k):
    c = Counter(words)
    ordered = sorted(c, key=lambda w: (-c[w], w))
    least = sorted(c, key=lambda w: (c[w], w))
    return ordered[:k], least[:k]


# BASE QUESTION (optimal): Counter plus two heaps of size
# K
# Idea: One pass counts the words. heapq.nsmallest with
# key (-count, word) gives the K most frequent and with
# key (count, word) the K least frequent, each in O(d log
# k) without a full sort.
def top_bottom(words, k):
    # ties are broken alphabetically
    c = Counter(words)
    top = heapq.nsmallest(k, c, key=lambda w: (-c[w], w))
    low = heapq.nsmallest(k, c, key=lambda w: (c[w], w))
    bottom = low
    return top, bottom


# FOLLOW-UP 1: The stream is too big to count every
# distinct word.
# Answer: Use the Space-Saving (or Misra-Gries) summary
# with m counters: a word that appears more than n / m
# times is guaranteed to be tracked, and its stored count
# is an upper bound.
class SpaceSaving:
    """Too many distinct words: m counters. A word seen
```

```
    over n / m times is always tracked."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, word):
        if word in self.counts:
            self.counts[word] += 1
        elif len(self.counts) < self.m:
            self.counts[word] = 1
        else:
            # replace the smallest counter; the new word
            # inherits its count + 1 (an upper bound)
```

```
            low = min(self.counts, key=self.counts.get)
            self.counts[word] = self.counts.pop(low) + 1

    def top_k(self, k):
        return sorted(self.counts, key=lambda w: (
            -self.counts[w], w))[:k]


# FOLLOW-UP 2: topK must be O(k), not O(d log k), because
# it is called all the time.
# Answer: Keep words in buckets by count (a linked list of
# count buckets, like the LFU cache structure); the most
# frequent words are in the last bucket. An increment
# moves a word one bucket up in O(1).
class FreqBuckets:
    """Stream: add(word) is O(1); top_k reads only the
    highest count buckets."""

    def __init__(self):
        self.count = defaultdict(int)
        # count -> words with exactly that count
        self.bucket = defaultdict(dict)
        self.high = 0

    def add(self, word):
        c = self.count[word]
        if c:
            # move the word up one bucket
            del self.bucket[c][word]
            if not self.bucket[c]:
                del self.bucket[c]
        self.count[word] = c + 1
        self.bucket[c + 1][word] = True
        self.high = max(self.high, c + 1)

    def top_k(self, k):
        out = []
        level = self.high
        while level > 0 and len(out) < k:
            # words inside one bucket: alphabetical
            out += sorted(self.bucket.get(level, ()))
            level -= 1
        return out[:k]

    def bottom_k(self, k):
```

```
        out = []
        for level in sorted(self.bucket):
            out += sorted(self.bucket[level])
            if len(out) >= k:
                break
        return out[:k]


# FOLLOW-UP 3: Case and punctuation.
# Answer: Normalise before counting (lowercase, strip
# punctuation) and say so; the counting and ranking code
# is unchanged.
def normalise(text):
```

```
    # case and punctuation: clean before counting
    letters = (ch for ch in text.lower() if ch.isalnum())
    return "".join(letters)


if __name__ == "__main__":
    words = "a b a c b a d d".split()
    top, bottom = top_bottom(words, 2)
    assert top == ["a", "b"] and bottom == ["c", "b"]
    f = FreqBuckets()
    for w in words:
        f.add(w)
    assert f.top_k(2) == ["a", "b"]
    assert f.bottom_k(2) == ["c", "b"]
    assert f.top_k(10) == ["a", "b", "d", "c"]
    s = SpaceSaving(2)
    for w in "x x x y z x x":
        s.add(w) if w.strip() else None
    assert s.top_k(1) == ["x"]
    assert normalise("Hello, World!") == "helloworld"
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], k: int) -> list[list[str]]:
```

Sample call

```
solution(['i', 'love', 'leetcode', 'i', 'love', 'coding'],
 2)
```

Test cases

Input	Expected
`(['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)`	`[['i', 'love'], ['coding', 'leetcode']]`
`(['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)`	`[['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]`